



St. Francis Institute of Technology

(Engineering College)

An Autonomous Institute, Affiliated to University of Mumbai NAAC

A+ Accredited | CMPN, EXTC, INFT NBA Accredited | ISO 9001:2015 Certified

Department of Artificial Intelligence and Machine Learning

Academic Year: 2025-2026 **Term:** Even (Jan. 2026 – Jun. 2026) **Class / Branch:** SE – AIML

Semester: IV

Course: Web Programming Lab. (AI4VS_LR4)

Date of Assignment: / /2026 **Date of Submission:** / /2026

Name : Sharon Elango **Roll no.:** 62 **Batch:** 3

Pre-Lab Exercises for Experiment-11

Part A: Understanding NoSQL and MongoDB

Task 1: Conceptual Preparation

Answer the following questions:

1. What is NoSQL?

- NoSQL stands for "Not Only SQL."
- It refers to non-relational database systems that store data in formats other than traditional tables.
- It is designed to handle unstructured data, high velocity, and massive scaling requirements.

2. How is NoSQL different from SQL databases?

- **Data Model:** SQL uses fixed tables and rows; NoSQL uses flexible documents, graphs, or key-value pairs.
- **Schema:** SQL requires a predefined, rigid schema; NoSQL allows for dynamic, evolving schemas.
- **Scaling:** SQL typically scales vertically (adding power to one server); NoSQL scales horizontally (adding more servers).
- **Transactions:** SQL emphasizes ACID compliance (reliability); NoSQL often prioritizes performance and availability.

3.What is MongoDB?

- MongoDB is a popular NoSQL, document-oriented database.
- It stores data in a JSON-like format, making it highly compatible with JavaScript applications.
- It is the "M" in the MERN stack.

4.What is a document in MongoDB?

- A document is the basic unit of data in MongoDB (similar to a row in SQL).
- It consists of field-value pairs stored in a binary format called BSON.
- Documents can have different structures even if they are in the same collection.

5.What is a collection?

- A collection is a grouping of MongoDB documents.
- It is the equivalent of a table in a relational database.
- Collections exist within a database and do not enforce a specific schema on the documents they contain.

6.What is BSON?

- BSON stands for "Binary JSON."
- It is a binary-encoded serialization of JSON-like documents used by MongoDB.
- It supports more data types than standard JSON, such as "Date" and "Binary Data."

7.What are advantages of MongoDB?

- **Flexibility:** No need to restart the database to change the data structure.
- **Speed:** High-speed indexing and query performance for document-based lookups.
- **Scalability:** Native support for "sharding" to distribute data across many machines.
- **Developer Friendly:** Uses JSON syntax which is easy for web developers to read and write.

Part B: MongoDB Practice (Without Express)

Task 2: Install and Explore MongoDB

1. Install MongoDB locally OR create a free account in **MongoDB Atlas**.
2. Create a new database named:
 - practiceDB
3. Create a collection:
 - students

Perform the following operations:

- Insert 3 student records manually.
- View inserted records.
- Delete one record.
- Update one record.

Submission:

Take screenshots of:

- Database creation
- Inserted documents
- Updated document

```
test> use practiceDB
switched to db practiceDB
```

```
practiceDB> db.students.insertMany([
|   { name: "Alice", email: "alice@sfit.com", age: 20 },
|   { name: "Bob", email: "bob@sfit.com", age: 21 },
|   { name: "Charlie", email: "charlie@sfit.com", age: 22 }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69b39bfe208ba0278d7c2907'),
    '1': ObjectId('69b39bfe208ba0278d7c2908'),
    '2': ObjectId('69b39bfe208ba0278d7c2909')
  }
}
practiceDB> |
```

```
practiceDB> db.students.find()
[
  {
    _id: ObjectId('69b39bfe208ba0278d7c2907'),
    name: 'Alice',
    email: 'alice@sfit.com',
    age: 20
  },
  {
    _id: ObjectId('69b39bfe208ba0278d7c2908'),
    name: 'Bob',
    email: 'bob@sfit.com',
    age: 21
  },
  {
    _id: ObjectId('69b39bfe208ba0278d7c2909'),
    name: 'Charlie',
    email: 'charlie@sfit.com',
    age: 22
  }
]
practiceDB> |
```

```
practiceDB> db.students.deleteOne({ name : " Charlie"})
{ acknowledged: true, deletedCount: 0 }
practiceDB>
```

```
practiceDB> db.students.updateOne({ name : "Alice"}, { $set : { age : 21 }})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
practiceDB>
```

Part C: Mongoose Familiarization

Task 3: Setup Basic Backend with Mongoose

1. Create a new Node.js project.
2. Install:
 - express
 - mongoose
3. Connect to MongoDB database.
4. Confirm successful connection in terminal.

```
PS C:\Users\HP> mkdir mongoose-practice

Directory: C:\Users\HP

Mode                LastWriteTime         Length Name
----                -
d-----          3/13/2026  10:56 AM                mongoose-practice

PS C:\Users\HP> cd mongoose-practice
PS C:\Users\HP\mongoose-practice> npm init -y
Wrote to C:\Users\HP\mongoose-practice\package.json:

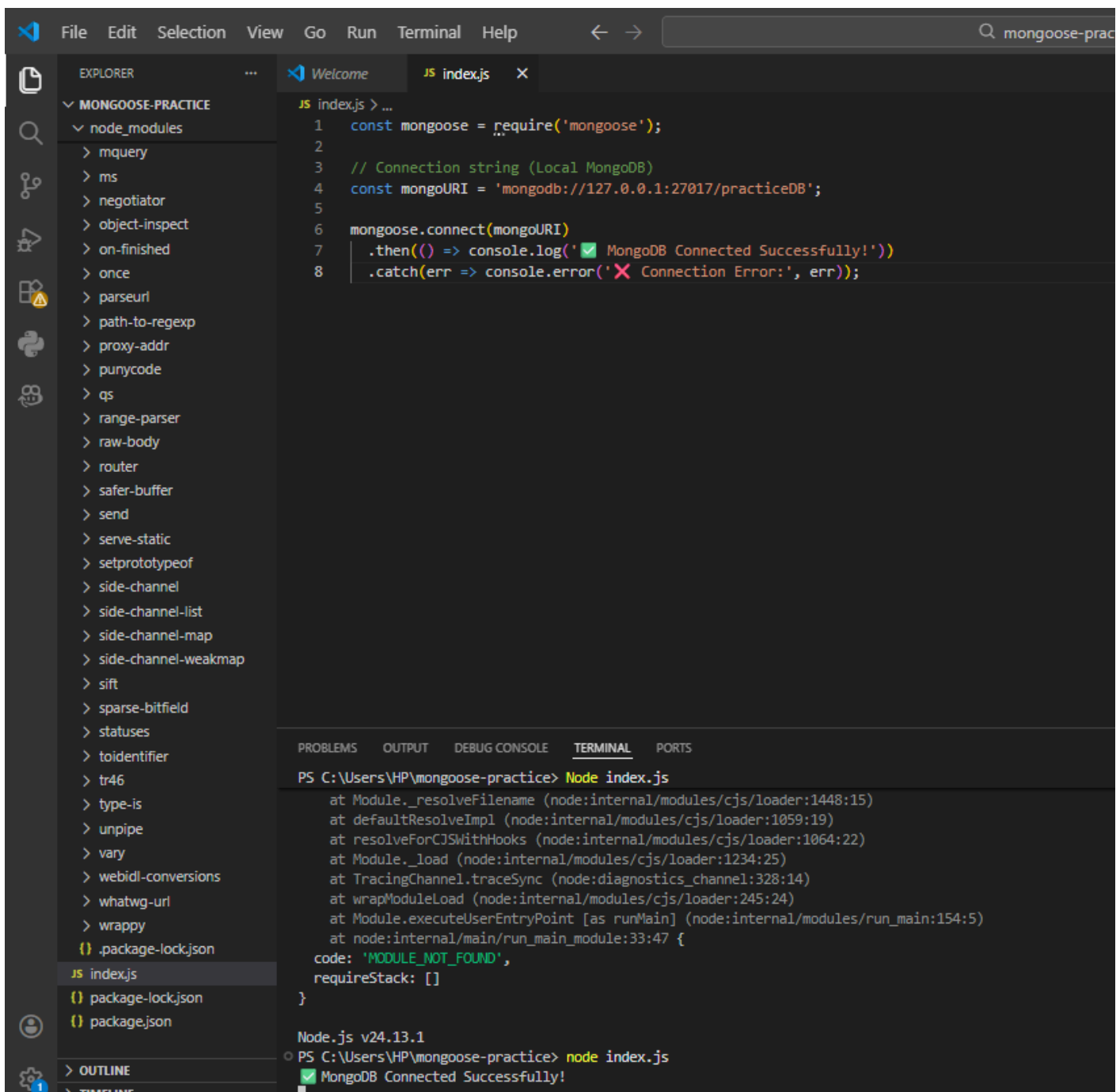
{
  "name": "mongoose-practice",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\HP\mongoose-practice> npm install express mongoose

added 82 packages, and audited 83 packages in 4s

23 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
PS C:\Users\HP\mongoose-practice> const mongoose = require('mongoose');
```



Questions:

What happens if the connection string is

wrong?

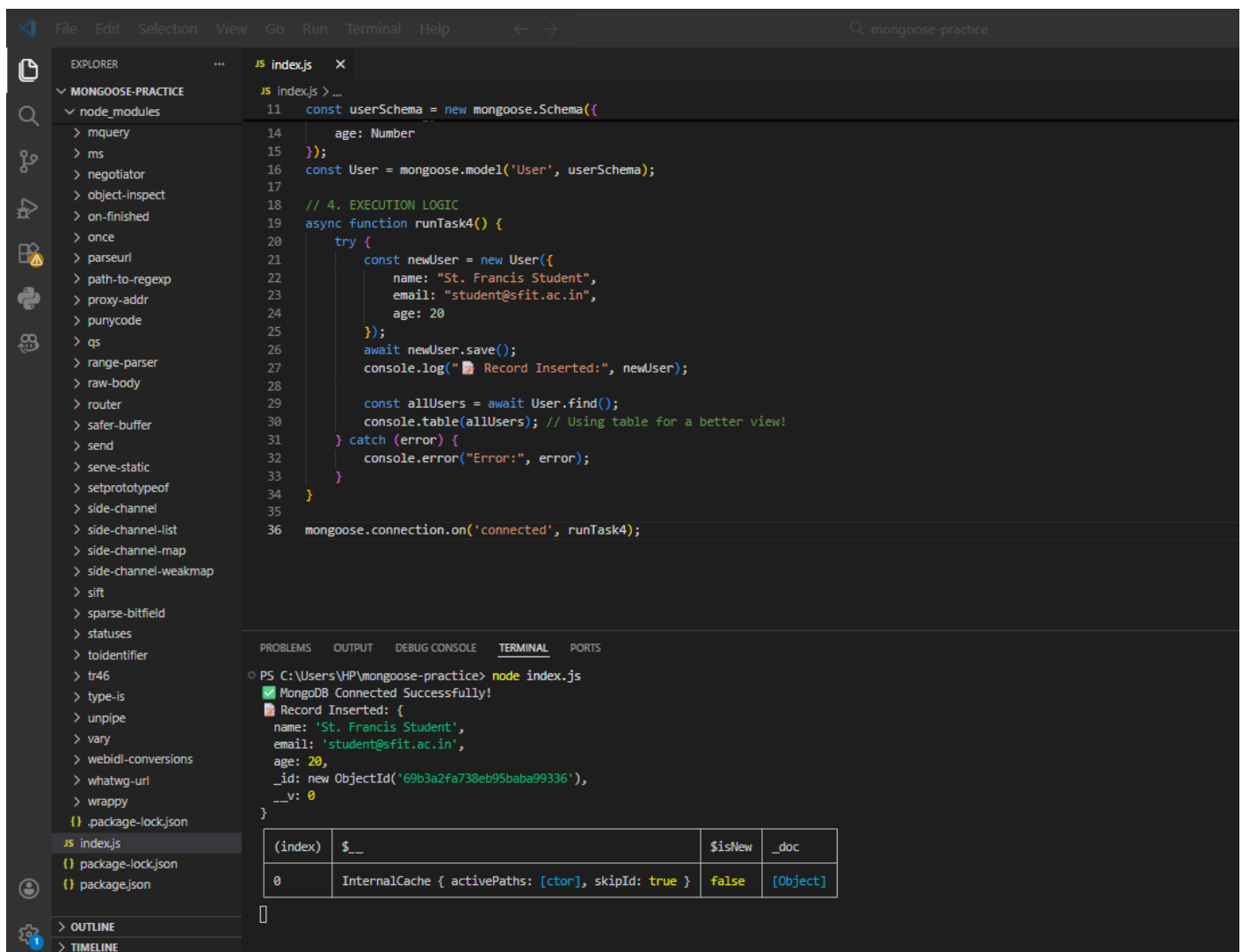
- The backend application will fail to establish a handshake with the database.
- The `mongoose.connect().catch()` block will trigger, printing an error message in the terminal.
- The server may remain running, but any API request that requires data will timeout or crash.

What is the purpose of Mongoose?

- It acts as an Object Data Modeling (ODM) library for MongoDB and Node.js.
- It provides a structured way to define schemas and interact with the database using JavaScript objects.
- It handles data validation, type casting, and simplifies complex queries.

Task 4: Create Schema and Model

1. Create a schema with fields:
 - name
 - email
 - age
2. Create a model using the schema.
3. Insert one record using the model.
4. Retrieve all records from collection.



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code in `index.js` defines a Mongoose schema for a user, creates a model, and includes logic to insert a new user and retrieve all users. The terminal output shows the successful connection to MongoDB, the insertion of a new user, and a table view of the retrieved users.

```
11 const userSchema = new mongoose.Schema({
14   age: Number
15 });
16 const User = mongoose.model('User', userSchema);
17
18 // 4. EXECUTION LOGIC
19 async function runTask4() {
20   try {
21     const newUser = new User({
22       name: "St. Francis Student",
23       email: "student@sfrit.ac.in",
24       age: 20
25     });
26     await newUser.save();
27     console.log("Record Inserted:", newUser);
28
29     const allUsers = await User.find();
30     console.table(allUsers); // Using table for a better view!
31   } catch (error) {
32     console.error("Error:", error);
33   }
34 }
35
36 mongoose.connection.on('connected', runTask4);
```

Terminal Output:

```
PS C:\Users\HP\mongoose-practice> node index.js
MongoDB Connected Successfully!
Record Inserted: {
  name: 'St. Francis Student',
  email: 'student@sfrit.ac.in',
  age: 20,
  _id: new ObjectId('69b3a2fa738eb95baba99336'),
  __v: 0
}
```

(index)	\$__	\$isNew	_doc
0	InternalCache { activePaths: [ctor], skipId: true }	false	[Object]

Observation:

- **Is the data stored in MongoDB?**
 - Yes, once a record is saved via the model, it is written to the physical database file on the disk.
- **Does schema restrict data structure?**
 - Yes, while MongoDB is schemaless, Mongoose enforces the structure at the application level.
 - Fields not defined in the schema are ignored when saving data to prevent "dirty" data.

•Part D: REST API Practice

Task 5: Create Basic REST APIs

Create the following routes:

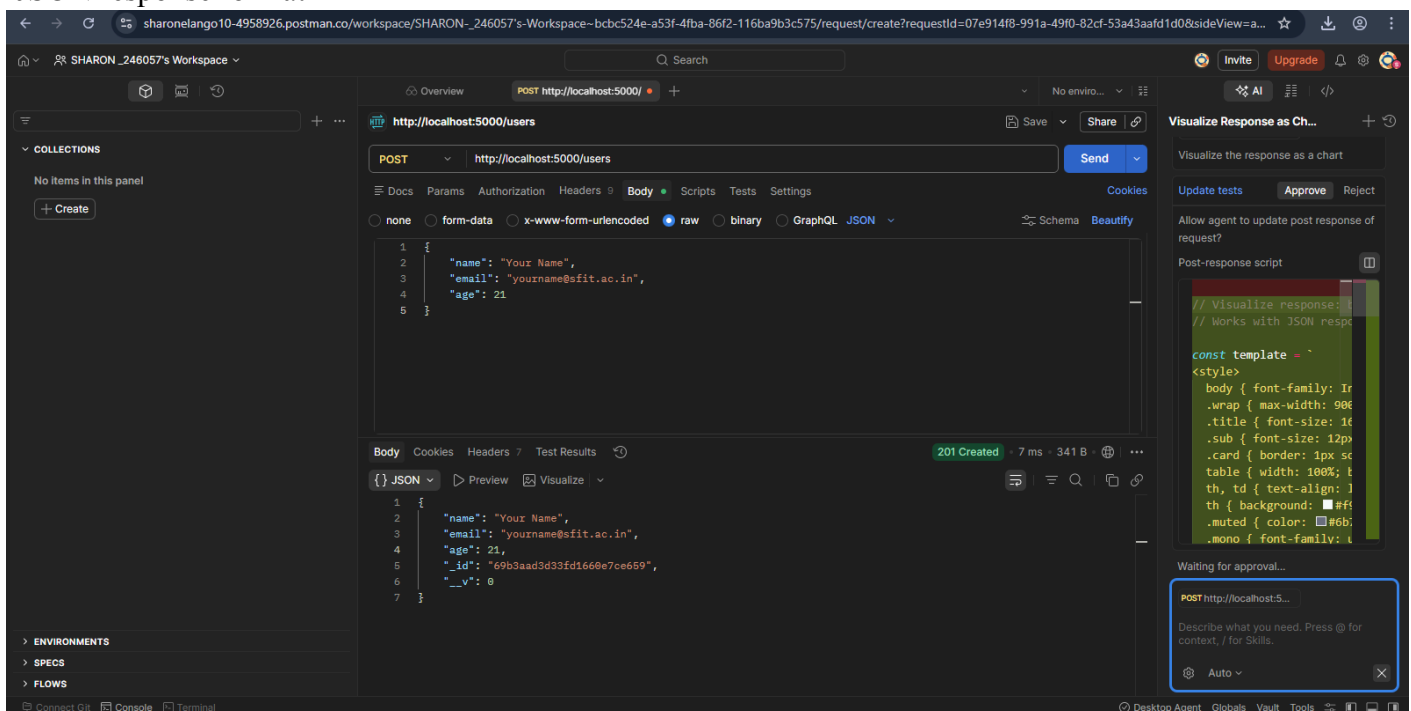
1. POST → Add user
2. GET → Retrieve all users

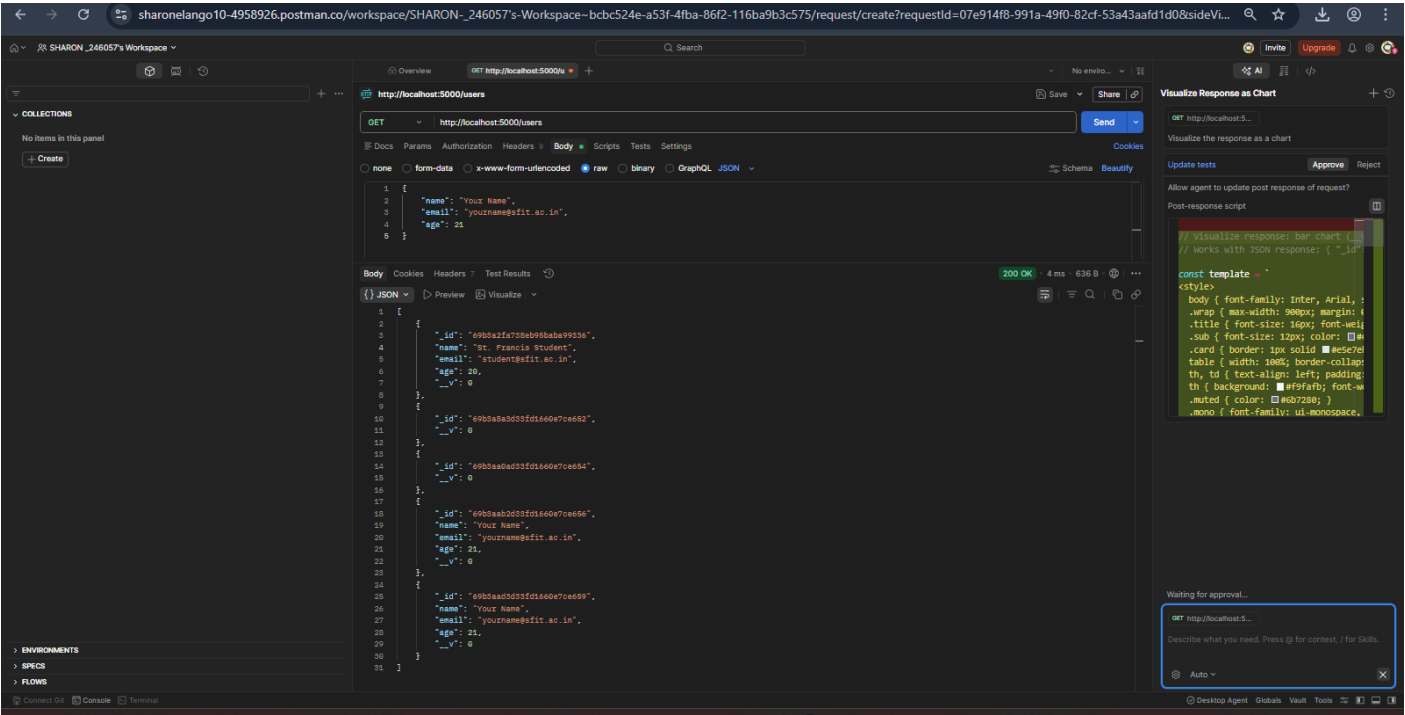
Test using:

- Postman

Verify:

- Status codes
- JSON response format





Part E: React API Practice

Task 6: Fetch Data in React

- 1. Create a simple React app.
- 2. Create a component:
 - o UserList
- 3. Fetch data from backend GET API.
- 4. Store data in state.
- 5. Display data using:
 - o List OR Table



Task 6: User Management System

Name	Email	Age
	s@sfit.ac.in	20
Sharon	s@sfit.ac.in	20
Sharon	s@sfit.ac.in	20
Sharon	s@sfit.ac.in	20
Persistence Test	test@sfit.com	25
Sharon	sharon@sfit.ac.in	20
Sharon	sharon@sfit.ac.in	20

Observation:

Does UI update after fetching data?

- Yes, because when the state (via `setUsers`) changes, React automatically re-renders the component to reflect the new data in the table.

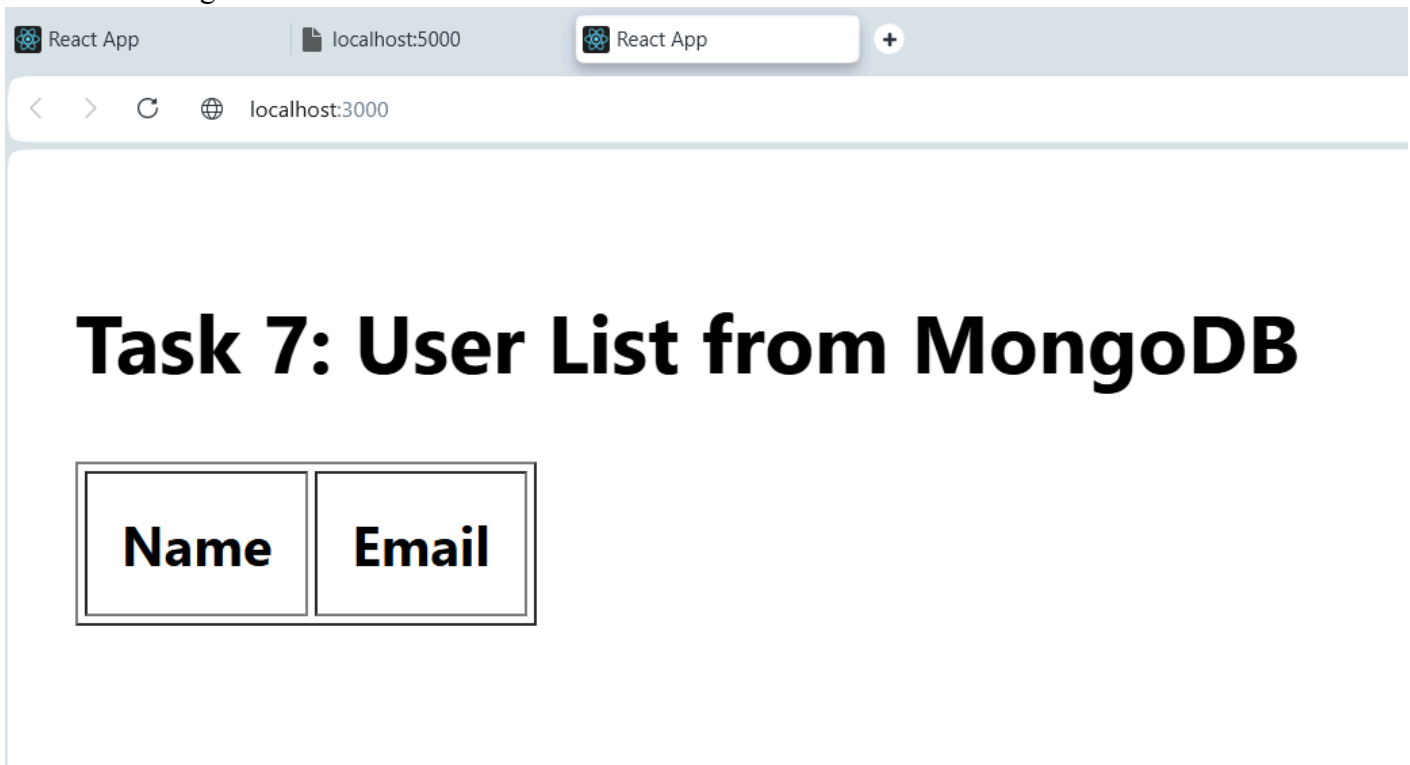
What happens if the backend is not running?

- The React `fetch` request will fail to connect to `localhost:5000`.
- The `.catch()` block in the React code will execute, and usually, an error message is shown to the user or logged in the browser console.

Part F: Data Persistence Practice

Task 7: Restart Server Test

1. Add user using POST request.
2. Restart backend server.
3. Fetch users again.



React App

localhost:5000

React App

+

<

>

↺

🌐

localhost:3000

Task 7: User List from MongoDB

Name	Email
	s@sfit.ac.in
Sharon	s@sfit.ac.in
Sharon	s@sfit.ac.in
Sharon	s@sfit.ac.in
Persistence Test	test@sfit.com

Question:

- Does data still exist? Why?

Yes, the data persists.

This is because the data is stored in the MongoDB database files on the hard drive, whereas the Node.js server only holds temporary data in the RAM.

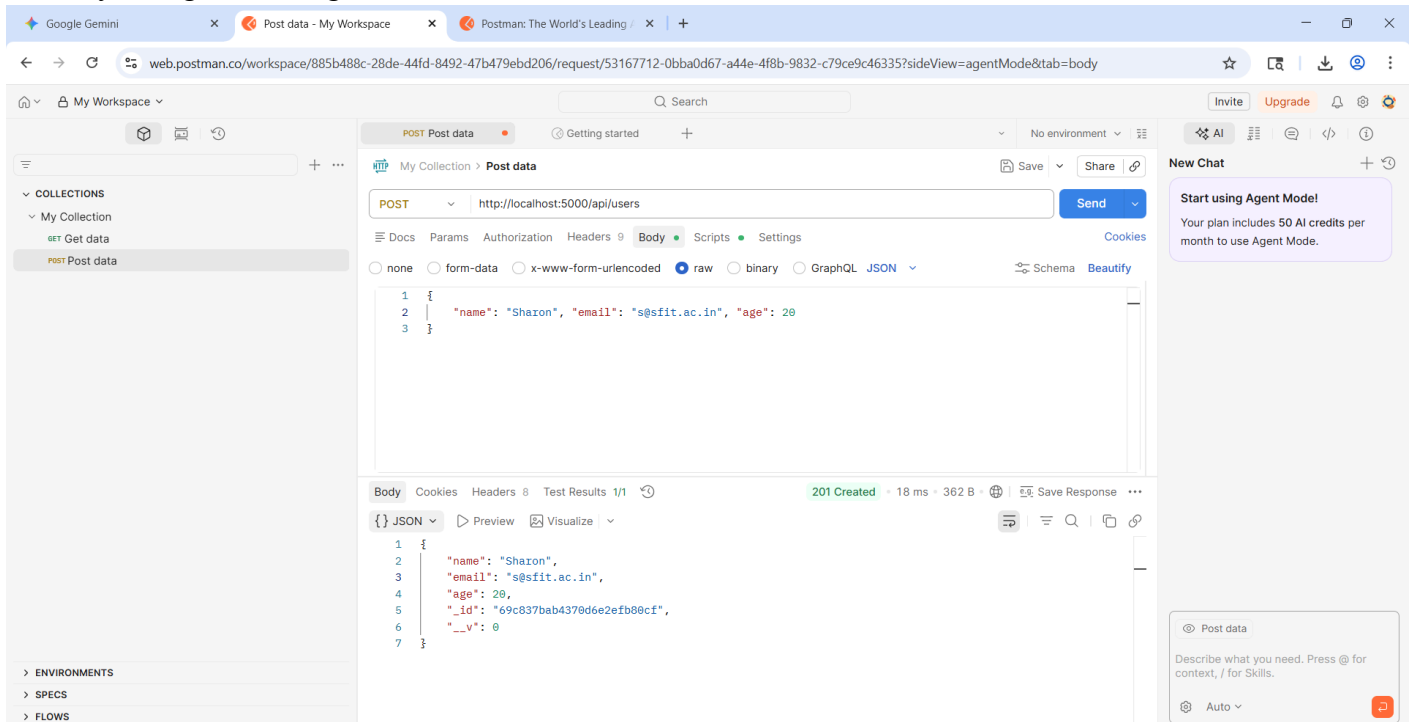
Task 8: Perform All CRUD Operations via API

Activities:

1. Extend the backend to include:
 - PUT → Update user by ID
 - DELETE → Delete user by ID
2. Test all four operations using Postman:

- Create user (POST)
- Retrieve users (GET)
- Update user (PUT)
- Delete user (DELETE)

3. Verify changes in MongoDB database.



Observations to Record:

What status codes are returned for each operation?

Standard REST API practices (which you implemented in your code) use these specific codes to tell the client the result of the request:

- **POST (Create):** Returns **201 Created**. This indicates that a new resource (user) was successfully generated in the database.
- **GET (Read):** Returns **200 OK**. This confirms the data was successfully retrieved and sent to the frontend.
- **PUT (Update):** Returns **200 OK**. This shows the existing record was found and modified successfully.
- **DELETE (Delete):** Returns **200 OK** or **204 No Content**. This confirms the record is no longer in the database.
- **Validation Error (Task 9):** Returns **400 Bad Request**. This tells the user their input (like age < 18) failed the rules.
- **Server Error:** Returns **500 Internal Server Error**. This happens if the MongoDB database is disconnected or the code crashes.

What happens if an invalid ID is used?

When you provide an ID that doesn't exist or is formatted incorrectly in a **PUT** or **DELETE** request:

- **Mongoose "Cast" Error:** If the ID is the wrong length or contains illegal characters, Mongoose triggers an error because it cannot "cast" that string into a valid MongoDB ObjectId.

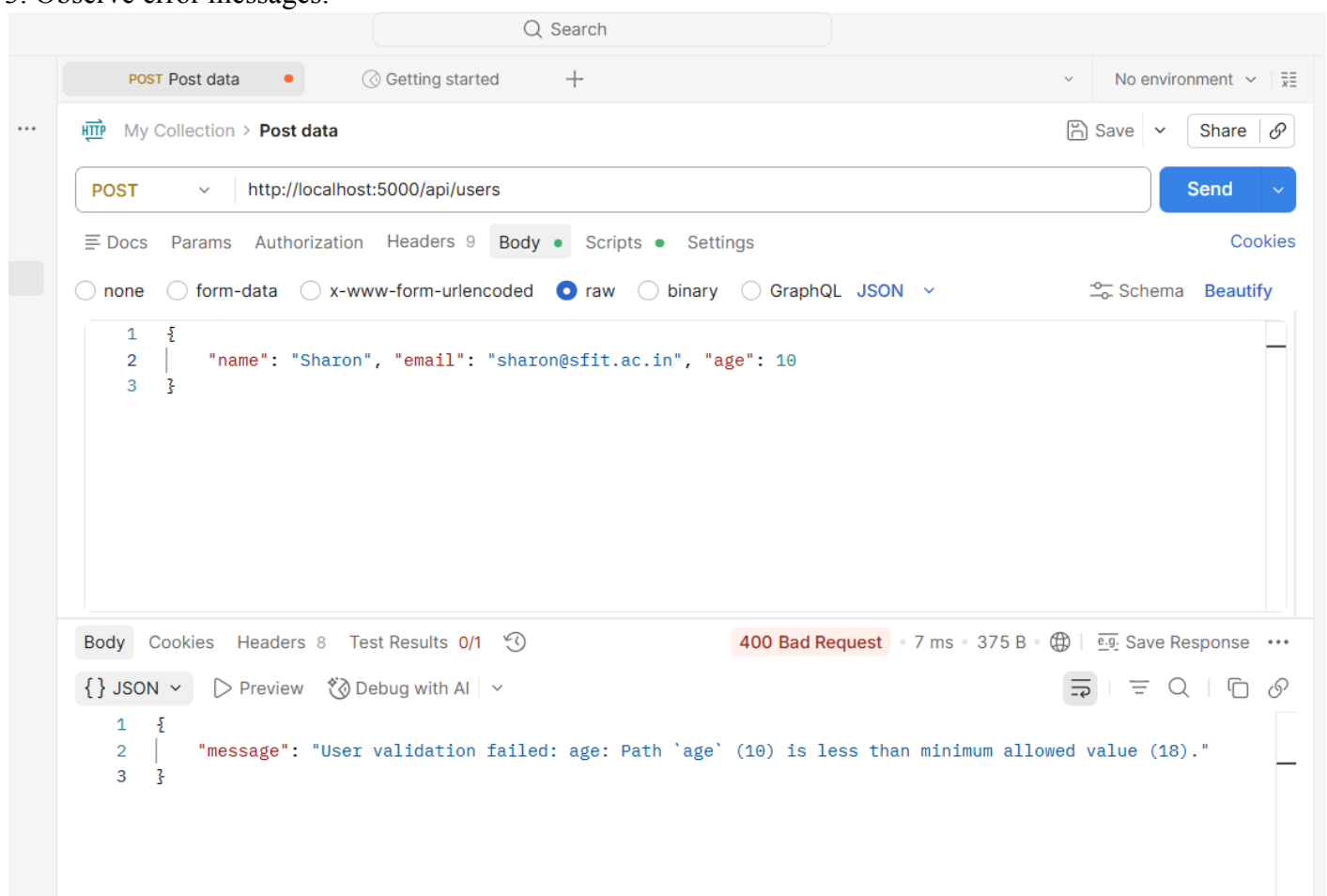
- **Graceful Catch:** Your `try...catch` block (Task 10) intercepts this error. Instead of the server crashing, it sends back a **400 Bad Request** or **404 Not Found** status code.
- **Null Result:** If the ID is formatted correctly but doesn't exist in the database, `findByIdAndUpdate` or `findByIdAndDelete` returns `null`. Your code then sends a "User not found" message.

Does the database reflect changes immediately?

- **Yes, changes are near-instantaneous.** MongoDB is designed for high-speed writes.
- **Atomic Operations:** As soon as the `await` keyword finishes in your Express route, the data has been physically committed to the database.
- **Frontend Sync:** While the database updates immediately, the **React UI** only reflects the change once you refresh the page or trigger a new `fetch()` call to the GET API.

Task 9: Implement Basic Validation in Mongoose Activities:

1. Modify schema to:
 - Make name required
 - Make email required
 - Add minimum age validation
2. Try inserting:
 - Valid data
 - Invalid data (missing fields or incorrect age)
3. Observe error messages.



The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5000/api/users
- Body (raw):**

```

1 {
2   "name": "Sharon", "email": "sharon@sfit.ac.in", "age": 10
3 }

```
- Status:** 400 Bad Request (7 ms, 375 B)
- Response (JSON):**

```

1 {
2   "message": "User validation failed: age: Path `age` (10) is less than minimum allowed value (18)."
3 }

```

Questions to Answer:

Does MongoDB allow invalid data without schema?

- **Yes, MongoDB is "schemaless" by nature.** At the database level, MongoDB does not care if one document has a "name" field and the next one doesn't.
- **Flexible Storage:** Without a tool like Mongoose, you could accidentally store a string in an "age" field or skip mandatory fields entirely.
- **Collection Level:** MongoDB treats documents as independent JSON-like objects, meaning it provides no built-in "Rules" to stop inconsistent data from being saved.

How does Mongoose validation improve data quality?

- **Acts as a Gatekeeper:** Mongoose sits between your Express code and the MongoDB database. It checks every piece of data against your **UserSchema** before allowing it through.
- **Enforces Constraints:** It ensures that "Required" fields are never left empty and that data fits the correct "Type" (e.g., ensuring Age is a Number, not text).
- **Business Logic:** It allows you to set custom rules, like the **min: 18** validation you implemented, which prevents logically incorrect data (like a minor signing up) from entering your system.
- **Data Consistency:** Because every "User" must follow the same schema, your frontend (React) can rely on the data always having the same fields, preventing "undefined" errors in your UI.

Task 10: Handle Errors Gracefully

Activities:

1. Add error handling in:
 - Database connection
 - POST route
 - GET route
2. Send incorrect requests from Postman:
 - Invalid ID
 - Missing fields
3. Return proper:
 - Status codes (400, 404, 500)
 - JSON error messages

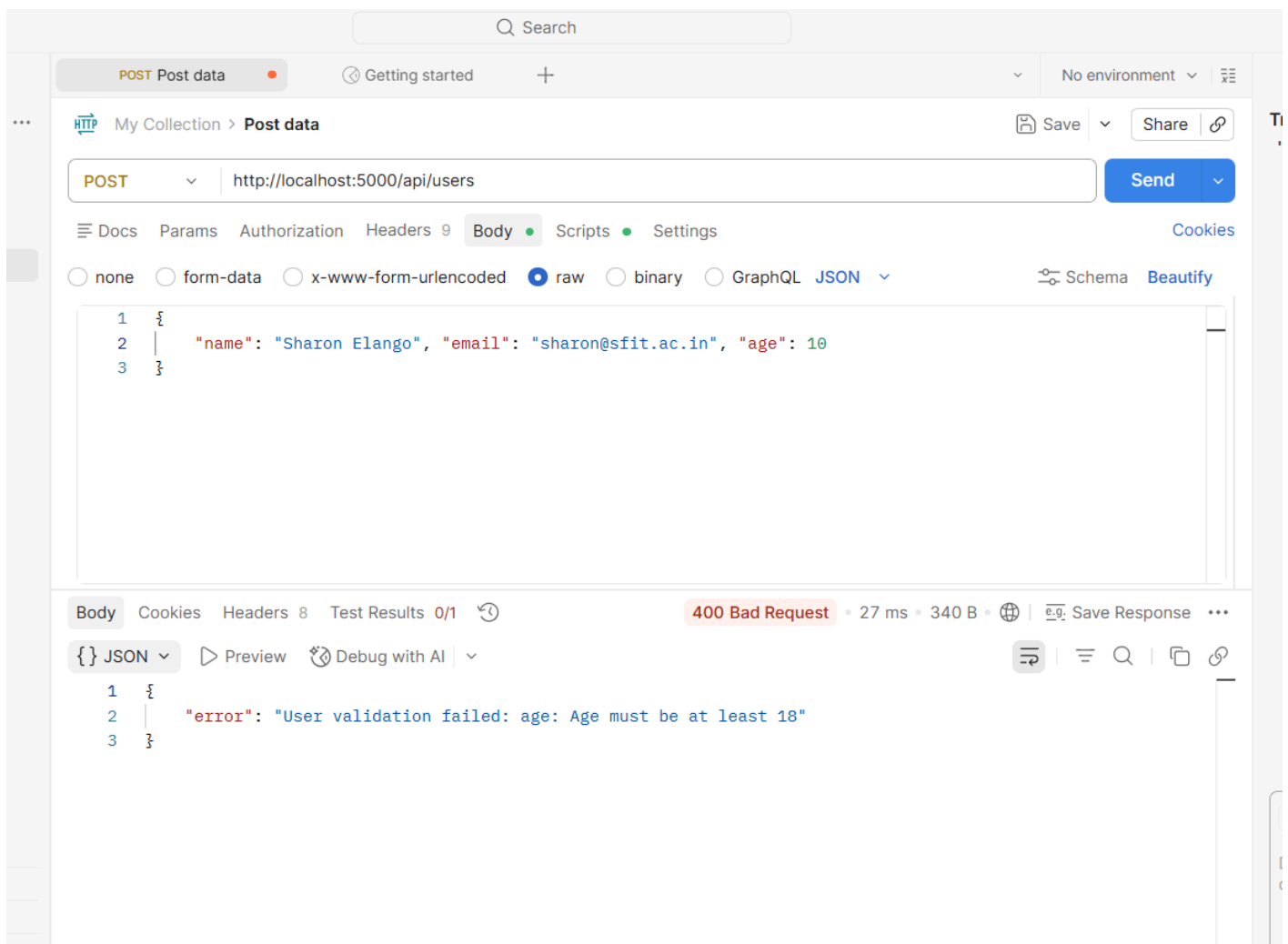
< > ↺ 🌐 localhost:3000

🔍 📄 🏠 🔄 ⏪ ⏩ ⚙️ 🌐 📄 🏠 🔄 ⏪ ⏩ ⚙️

MERN Stack User List

Could not connect to server.

Name	Email	Age
------	-------	-----



Observations:

• What status code is returned for bad requests?

- **400 Bad Request:** This is the standard status code returned when the server cannot process the request due to client-side errors.
- **Validation Failures:** In your project, this is triggered when a user sends data that violates the Mongoose schema (e.g., age is less than 18 or a required name is missing).
- **Malformed JSON:** It is also returned if the JSON body sent via Postman has syntax errors (like a missing comma or quote).

What happens if MongoDB server is stopped?

- **Connection Timeout:** The Express backend will attempt to reach the database and eventually fail, triggering the `.catch()` block in your database connection code.
- **500 Internal Server Error:** Any API route (GET, POST, etc.) that tries to query the database will fail. Because of your error handling, the server will return a **500 status code** to the client instead of simply hanging indefinitely.
- **Console Errors:** Your Node.js terminal will display a "MongoServerError: ServerSelectionError," indicating that the "heart" of the data storage is unreachable.
- **Frontend Impact:** The React app will receive an error response and, based on your `UserList.js` logic, will display an error message like "Could not connect to server" rather than displaying an empty table or crashing the browser.

How does proper error handling improve API reliability?

- **Prevents Server Crashes:** Without `try...catch` blocks, a single invalid request could crash the entire Node.js process, taking the website offline for everyone.
- **Predictable Behavior:** It ensures the API always sends a response. Even if it's an error message, a response allows the frontend to know exactly what went wrong and react accordingly.
- **Better Debugging:** Detailed JSON error messages (like "Name is mandatory") help developers identify exactly which field caused the issue without digging through thousands of lines of server logs.
- **Security:** Proper error handling allows you to send "safe" messages to the user while logging the "ugly" technical details privately on the server, preventing attackers from seeing your database structure.
- **User Experience:** It allows the frontend to show helpful hints to the user (e.g., "Please enter an age above 18") rather than leaving them staring at a broken page.